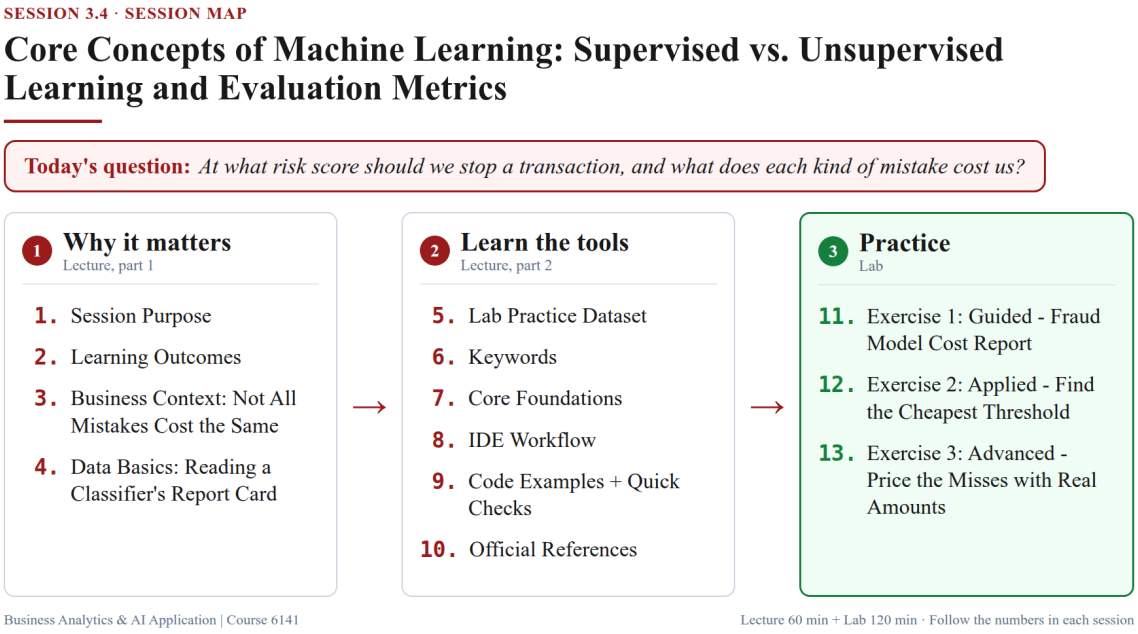
Session 3.4: Core Concepts of Machine Learning: Supervised vs. Unsupervised Learning and Evaluation Metrics

Course 6141: Business Analytics & AI Application | Reading for Week 12



Schedule: Week 12 | In-person classes (1:2 Delivery Ratio: 60 Min Lecture + 120 Min Lab)

1. Session Purpose

- A card issuer's fraud model gives every transaction a risk score from 1 to 100; above a cut-off score (the threshold), the transaction is held and the customer gets a verification text
- The two possible mistakes have very different costs: a false alarm costs about $5 of checking, while a missed fraud costs the stolen money
- The fraud operations manager asks: at what risk score should we stop a transaction, and what does each kind of mistake cost us? Today, you judge a model by business cost, not by a single accuracy number

2. Expected Learning Outcomes (By the end of this session, you will be able to)

1. "Which kind of learning fits this problem?": tell supervised learning (learning from examples with known answers) from unsupervised learning (finding groups without answers), and group transactions with K-Means
2. "How good is the risk score, really?": read a confusion matrix (a 2x2 count of right and wrong decisions) and calculate precision, recall, and ROC-AUC
3. "Where should we set the threshold?": choose the threshold with the lowest business cost, and fix the `ValueError` raised when scores are passed in place of 0/1 decisions

3. Business Context: Not All Mistakes Cost the Same

- Every decision on a transaction lands in one of four boxes:

	Model says: fraud (hold it)	Model says: legitimate (let it pass)
Really fraud	TP (True Positive): fraud caught	FN (False Negative): fraud missed, the money is lost
Really legitimate	FP (False Positive): false alarm, about $5 to check	TN (True Negative): normal purchase, no cost

- In our cost model, a missed fraud costs `$1,500` and a false alarm costs `$5`, so one miss costs as much as 300 false alarms.
- The accuracy trap: if only 1% of transactions are fraud, a model that says "never fraud" is 99% accurate and catches nothing. Accuracy hides the costly mistakes.

4. Data Basics: Reading a Classifier's Report Card

- Four measures, each answering one question:

Measure	Formula	Question it answers
Accuracy	(TP + TN) / all	What share of all decisions was right?
Precision	TP / (TP + FP)	Of the transactions we held, what share was really fraud?
Recall	TP / (TP + FN)	Of all the fraud, what share did we catch?
ROC-AUC	area under the ROC curve, from 0.5 to 1.0	How well does the score rank fraud above legitimate transactions, at every threshold?

- A higher threshold holds fewer transactions: fewer false alarms (higher precision) but more missed fraud (lower recall). The threshold moves this trade-off; the business cost decides where it should sit.

- ROC-AUC (Area Under the Receiver Operating Characteristic Curve): `0.5` means the score ranks no better than a coin flip; `1.0` means every fraud scores above every legitimate transaction.
- Supervised vs unsupervised:
  - **Supervised:** past examples come with the answer (`Actual_Fraud_Binary`), and the model learns to predict it. Today's risk score is a supervised model's output.
  - **Unsupervised:** no answer column; the method finds groups of similar rows by itself. K-Means is the most common example.

5. Lab Practice Dataset

- File Link: `data/session_03_04_dataset.csv`
- Audited 16-row card transaction dataset (fictional teaching data), one row per transaction, checked afterwards by fraud investigators:
  - `Transaction_ID`: Transaction code (e.g., `F-TXN-8001`).
  - `Transaction_Amount_USD`: Purchase amount in U.S. Dollars.
  - `Foreign_IP_Flag`: `1` if the purchase came from a foreign IP (Internet Protocol) address, the network address of the device; `0` if domestic.
  - `Failed_Login_Attempts_Count`: Failed login attempts before the purchase.
  - `Cardholder_Risk_Score`: The fraud model's risk score, from 1 (safe) to 100 (very risky).
  - `Actual_Fraud_Binary`: The investigators' answer: `1` = fraud, `0` = legitimate. 7 of the 16 transactions were fraud.

6. Keywords (Core Programming Concepts)

- `Supervised vs Unsupervised Learning`:
  - What: learning with known answers vs finding groups without answers
  - Why: the first question to ask about any ML project
- `Threshold`:
  - What: the cut-off score above which the model's answer becomes "yes"
  - Why: the business sets it, not the model
- `Confusion Matrix`:
  - What: the 2x2 count of TP, FP, FN, and TN
  - Why: shows each kind of mistake separately, so each can be priced
- `Precision` and `Recall`:
  - What: the share of holds that were fraud, and the share of fraud that was held
  - Why: the two sides of the trade-off a threshold controls
- `ROC-AUC`:
  - What: one number for how well a score ranks positives above negatives
  - Why: compares scoring models before any threshold is chosen

- `K-Means Clustering`:
  - What: an unsupervised method that splits rows into K groups of similar values
  - Why: finds customer or transaction segments when no labels exist

7. Core Programming & Computational Foundations

- From scores to decisions: `(df["Cardholder_Risk_Score"] >= 70).astype(int)` turns each score into `1` (hold) or `0` (pass). Metrics such as `confusion_matrix()` need these 0/1 decisions.
- Passing the raw scores instead raises `ValueError: Classification metrics can't handle a mix of binary and continuous targets`: the answers are 0/1, but the scores are continuous numbers.
- `confusion_matrix(y_true, y_pred).ravel()` flattens the 2x2 table into four numbers in the order `tn, fp, fn, tp`, so `tn, fp, fn, tp = ...` names them in one line.
- `roc_auc_score(y_true, scores)` is the exception: it takes the scores themselves, because it checks every threshold at once.
- K-Means needs scaled features (Session 3.2): otherwise the dollar amount, with its large numbers, would decide the groups on its own.

8. Development Workflow in Google Antigravity IDE

- `[1]` File Explorer: create your script with your name and the session code (for example, `Reacher_S34_demo_1.py`), and check the dataset is in `data/`.
- `[2]` Code Editor: write a few lines, run, repeat. Small steps keep errors small.
- `[3]` AI Agent Chat: on an error, capture the traceback (Python's error report) with `Win + Shift + S` (macOS: `Cmd + Shift + 4`), paste it with `Ctrl + V`, and ask why it failed.
- `[4]` Terminal: run `python` followed by your script name. Return code `0` means the script finished with no errors. First time only: `pip install scikit-learn`.

9. Hands-On Coding Micro-Examples

- Every example has three parts: **1. Input**, **2. Process**, and **3. Output**. Name each file with your own name and the session code, for example `Reacher_S34_demo_1.py` (pattern: `YourName_S34_demo_1.py`).
- *Micro-Example 1 (Confusion Matrix at Threshold 70):* `YourName_S34_demo_1.py`

```python
# 1. Input: the investigators' answers and the model's scores
import pandas as pd
from sklearn.metrics import confusion_matrix, precision_score, recall_score
df = pd.read_csv("data/session_03_04_dataset.csv")
y_true = df["Actual_Fraud_Binary"]

# 2. Process: hold every transaction scoring 70 or more
y_pred = (df["Cardholder_Risk_Score"] >= 70).astype(int)
cm = confusion_matrix(y_true, y_pred)
precision = precision_score(y_true, y_pred)
```

```
recall = recall_score(y_true, y_pred)

# 3. Output: the 2x2 table, then the two key measures
print("Rows: really legitimate, really fraud | Columns: passed, held\n", cm)
print(f"Precision: {precision:.3f} | Recall: {recall:.3f}")
```

  - `.astype(int)` turns `True` / `False` into `1` / `0`.
  - Result: `[[8 1] [0 7]]`: 8 legitimate passed, 1 false alarm (`F-TXN-8009`, score `71`), 0 frauds missed, 7 frauds held. Precision `0.875`, recall `1.000`.

- *Quick Check 1 (`classification_report()`):*
  - New tool: `classification_report(y_true, y_pred)` prints precision, recall, and the F1-score (one number that balances precision and recall) for both classes, plus accuracy. Why: the full report card in one call.
  - Task: add `classification_report` to the import line, then add one line to part **3. Output**: `print(_____(y_true, y_pred))`
  - Check: the `accuracy` line shows `0.94`, and the row for class `1` shows precision `0.88` and recall `1.00`.

- *Micro-Example 2 (ROC-AUC: Ranking Quality at Every Threshold):* `YourName_S34_demo_2.py`

```
# 1. Input: the answers and the scores
import pandas as pd
from sklearn.metrics import roc_auc_score
df = pd.read_csv("data/session_03_04_dataset.csv")
y_true = df["Actual_Fraud_Binary"]

# 2. Process: AUC uses the scores directly, not 0/1 decisions
auc = roc_auc_score(y_true, df["Cardholder_Risk_Score"])

# 3. Output: compare the score ranges of the two groups
print(f"ROC-AUC: {auc:.3f}")
print(df.groupby("Actual_Fraud_Binary")["Cardholder_Risk_Score"].agg(["min",
 ↪  "max"]))
```

  - `.agg(["min", "max"])` returns the smallest and largest score in each group.
  - Result: ROC-AUC `1.000`. Every fraud scored between `74` and `92`, every legitimate transaction between `12` and `71`, so some threshold separates them perfectly. With only 16 transactions, treat this as encouraging, not as proof.

- *Quick Check 2 (`accuracy_score()`):*
  - New tool: `accuracy_score(y_true, y_pred)` returns the share of correct decisions. Why: shows how a single accuracy number can look good while hiding the costly mistakes.
  - Task: add two lines to part **3. Output**: `from sklearn.metrics import accuracy_score` and `print(_____(y_true, (df["Cardholder_Risk_Score"] >= 90).astype(int)))`

  - Check: the Terminal shows `0.625`. At threshold 90, 62.5% of the decisions are right, yet 6 of the 7 frauds pass through.

- *Micro-Example 3 (K-Means: Groups Without Answers):* `YourName_S34_demo_3.py`

```
# 1. Input: two features only; the fraud column is NOT used
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
df = pd.read_csv("data/session_03_04_dataset.csv")
features = df[["Transaction_Amount_USD", "Cardholder_Risk_Score"]]

# 2. Process: scale, then form 3 groups of similar transactions
scaled = StandardScaler().fit_transform(features)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df["Cluster_ID"] = kmeans.fit_predict(scaled)

# 3. Output: the average profile of each group, and its fraud share
profile = df.groupby("Cluster_ID")[
    ["Transaction_Amount_USD", "Cardholder_Risk_Score", "Actual_Fraud_Binary"]
].mean().round(1)
print(profile)
```

  - `n_clusters=3` asks for 3 groups; `n_init=10` tries 10 starting points and keeps the best; `random_state=42` makes the result repeatable.
  - `.fit_predict()` learns the groups and returns each row's group number.
  - Result: one group of small, low-risk purchases (average `$52.2`, score `16.1`, `0.0` fraud), one of mid-size, higher-risk purchases (`$1,795.0`, `74.8`, `0.8` fraud), and one of large, high-risk purchases (`$3,587.5`, `88.5`, `1.0` fraud). K-Means never saw the fraud column, yet its groups line up with it.

- *Quick Check 3 (`.size()`):*

  - New tool: `df.groupby("Column").size()` counts the rows in each group. Why: a segment's size matters as much as its profile.
  - Task: add one line to part **3. Output**: `print(df.groupby("Cluster_ID").____())`
  - Check: group sizes `4`, `8`, and `4`.

10. Official Python Documentation & References

- Scikit-Learn: Classification Metrics (Confusion Matrix, Precision-Recall).
- Scikit-Learn: Clustering (K-Means Algorithm Overview).
- Fawcett: An Introduction to ROC Analysis (Pattern Recognition Letters).

11. Exercise 1: Guided Practice - Fraud Model Cost Report

- *Step 1: Open Script:* In Google Antigravity IDE, create a new script named `YourName_S34_evaluation.py`, replacing `YourName` with your own name (for example, `Reacher_S34_evaluation.py`).

- *Step 2: Count the Outcomes and Price the Mistakes:* Type the following code:

```
# 1. Input: the answers and the scores
import pandas as pd
from sklearn.metrics import confusion_matrix, precision_score, recall_score
df = pd.read_csv("data/session_03_04_dataset.csv")
y_true = df["Actual_Fraud_Binary"]

# 2. Process: decisions at threshold 70, then the cost of the mistakes
y_pred = (df["Cardholder_Risk_Score"] >= 70).astype(int)
tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
false_alarm_cost = 5.0
missed_fraud_cost = 1500.0
total_cost = fp * false_alarm_cost + fn * missed_fraud_cost

# 3. Output: counts, measures, and the cost
print(f"TP={tp}, FP={fp}, FN={fn}, TN={tn}")
print(f"Precision={precision_score(y_true, y_pred):.2f}, "
      f"Recall={recall_score(y_true, y_pred):.2f}")
print(f"Cost of mistakes: ${total_cost:,.2f}")
```

  - Two strings written next to each other inside `print()` are joined into one; this keeps long lines readable.

- *Step 3: Interpret:* In two comment lines, explain why one missed fraud costs as much as 300 false alarms, and what that means for where the threshold should sit.
- *Step 4: Execute in Terminal:* Run `python YourName_S34_evaluation.py` in Panel `[4]` and confirm return code `0`.
- *Success Check:* `TP=7, FP=1, FN=0, TN=8`; `Precision=0.88, Recall=1.00`; `Cost of mistakes: $5.00`.

12. Exercise 2: Applied Practice - Find the Cheapest Threshold

- *Coding Challenge:* In `YourName_S34_practice.py`, test thresholds from 50 to 90 in steps of 5 and find the one with the lowest cost.
- *Task 1 (Threshold Loop):* Loop with `for threshold in range(50, 95, 5):`. For each threshold, compute the decisions, `tn, fp, fn, tp`, and the cost (false alarm `$5`, missed fraud `$1,500`). Append one dictionary per threshold to a list, and turn the list into a DataFrame with `Threshold`, `TP`, `FP`, `FN`, `TN`, and `Cost_USD`. `range(50, 95, 5)` gives 50, 55, …, 90; the stop value 95 itself is not included.
- *Task 2 (Read the ValueError & AI Diagnosis):* Inside the loop, try `confusion_matrix(y_true, df["Cardholder_Risk_Score"] / 100)`, passing the scores instead of the decisions. The script stops with `ValueError: Classification metrics can't handle a mix of binary and continuous targets`. Capture the traceback with `Win + Shift + S`, paste it into Panel `[3]` (AI Agent Chat) with `Ctrl + V`, and ask how to turn scores into decisions.

- *Task 3 (Choose and Save):* Fix the call with `(score >= threshold).astype(int)`. Choose the threshold with the lowest cost; if several tie, choose the highest of them, because it sends the fewest customers to verification on future transactions. Save the table to `data/fraud_threshold_costs.csv` with `index=False`.
- *Success Check:* thresholds 50 to 70 cost `$5` (1 false alarm); 75 costs `$1,500`; 80 and 85 cost `$4,500`; 90 costs `$9,000`. Chosen threshold: `70`.

13. Exercise 3: Advanced Practice - Price the Misses with Real Amounts

- *Goal:* The flat `$1,500` per missed fraud is a guess. Replace it with each missed transaction's real amount, test every possible threshold, and show the manager how much the guess was off.
- *What you need:*
  - File: `data/session_03_04_dataset.csv` (described in **5. Lab Practice Dataset**).
  - Script name: `YourName_S34_challenge.py`.
  - Tools you already know: Boolean masks with `&` and `~` (Session 1.4), `.sum()`, `for` loops, `range()`, lists of dictionaries, and `pd.DataFrame()`.
- *Requirements (follow them in order):*
  1. For every whole-number threshold from 1 to 100, mark the held transactions: `held = df["Cardholder_Risk_Score"] >= threshold`.
  2. Compute two costs for each threshold:
     - `Flat_Cost_USD`: false alarms x `$5` + missed frauds x `$1,500`
     - `Real_Cost_USD`: false alarms x `$5` + the **sum of `Transaction_Amount_USD`** of the missed frauds
  3. Put all 100 results in one DataFrame with `Threshold`, `False_Alarms`, `Missed_Frauds`, `Flat_Cost_USD`, and `Real_Cost_USD`.
  4. Print every threshold with the lowest `Real_Cost_USD`.
  5. Print the rows for thresholds `70`, `75`, `80`, and `90`, and add `Gap_USD` = `Real_Cost_USD` - `Flat_Cost_USD`.
  6. In two comment lines, recommend a threshold and explain why the flat guess is risky.
- *New tools for this exercise:*
  - A false alarm is a held legitimate transaction: `held & (df["Actual_Fraud_Binary"] == 0)`. A missed fraud is a passed fraud: `~held & (df["Actual_Fraud_Binary"] == 1)`. `~` flips `True` and `False`.
  - Summing money over a mask: `df.loc[missed, "Transaction_Amount_USD"].sum()` adds up the amounts of the missed frauds only.
  - Selecting several rows by value: `results[results["Threshold"].isin([70, 75, 80, 90])]`.
- *Hints:*
  - Inside the loop, compute the two masks first, then both costs, then append one dictionary.
  - `range(1, 101)` gives 1, 2, …, 100.
- *Self-Check (expected results):*

- Lowest real cost: `$0` at thresholds `72`, `73`, and `74`: every fraud held and no false alarm.
- Gaps: threshold 70: `$0` (`$5` both ways); 75: `$600` (`$1,500` flat vs `$2,100` real); 80: `$830` (`$4,500` vs `$5,330`); 90: `$6,480` (`$9,000` vs `$15,480`).
- Business meaning: the missed frauds at high thresholds are the large ones, so the flat guess understates the loss more and more as the threshold rises. On these 16 past transactions the safest choice is `72` to `74`, but a perfect result on so few rows will not last; re-check the threshold every month on new investigated cases, and keep using real amounts in the cost.